PORTFOLIO

WEB DEVELOPER PORTFOLIO

스마트팜 IoT 웹&앱 개발

이근형

ABOUT ME

자기소개

안녕하세요, 신입 개발자 이근형입니다.
진득하게 몰입하여 문제를 끝까지 해결하는 개발자입니다.
일본어 전공에서 언어의 논리를 배우고, 프로그래밍 언어로 그 논리를 구현하며
'언어'라는 공통점 속에서 개발의 즐거움을 발견했습니다.
매일 학습을 기록하고, 팀과 협업하며 성장하는 개발자가 되겠습니다.

Education & Training

2025.02.14
울산대학교 일본어일본학과 졸업

2025.05.28 - 2025.11.20
IoT 지능형 관리시스템 웹&앱 개발자 양성과정
(Java, Python, React) 수료

Contact

Tel: 010-4792-3663

Email: geunhyeong930@gmail.com

GitHub: 클릭해 주세요.

Notion: 클릭해 주세요.

SKILLS

BACK-END

Java

spring

FRONT-END

HTML

CSS

JS

React

DATABASE

MariaDB

ETC

PROJECT

React 기반 스마트축사 IoT 기기 연동 및 가축 구매/판매몰 구축

Rest API, React 기반 IoT 통합 관리 시스템

OVERVIEW

프로젝트 개요

React, React Native, Spring Boot를 활용한 IoT 기반 축사 환경 관리 앱과 쇼핑몰 웹사이트 개발 팀 프로젝트입니다.
실시간 데이터 수집과 시각화, 사용자 인증, 권한 관리 등 다양한 기능을 구현하며
프론트엔드와 백엔드 개발 경험을 쌓았습니다.

프로젝트 진행 기간

1차 | 2024.09.11 ~ 2024.09.24

Raspberry Pi + Python 기반 스마트축사 IoT 기기 개발, 센서 데이터 수집 및 자동제어 알고리즘 구현

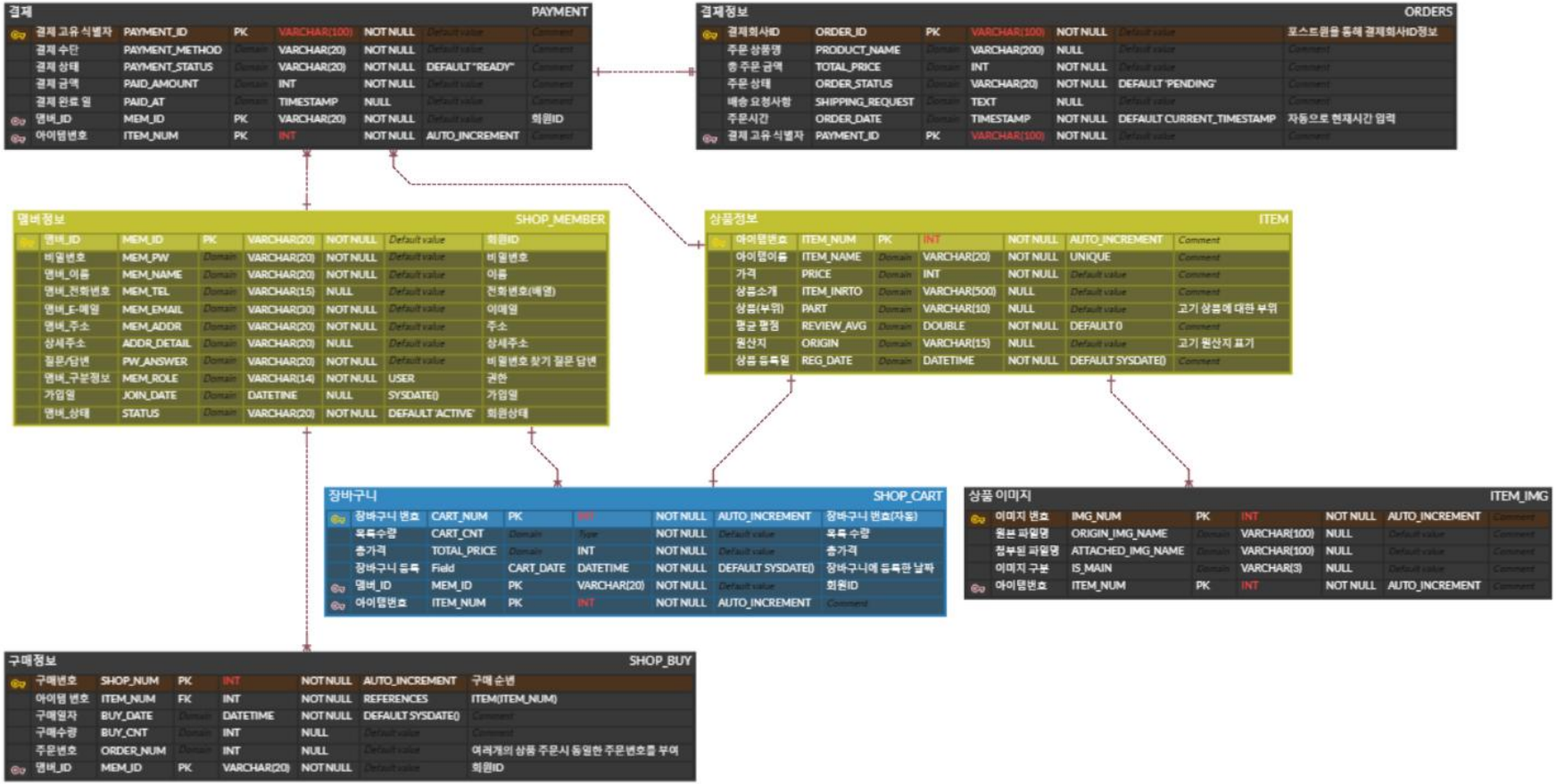
2차 | 2024.09.25 ~ 2024.10.16

React 기반 스마트축사 IoT 기기 연동, 가축 구매/판매몰 웹사이트 구축

3차 | 2024.10.17 ~ 2024.11.14

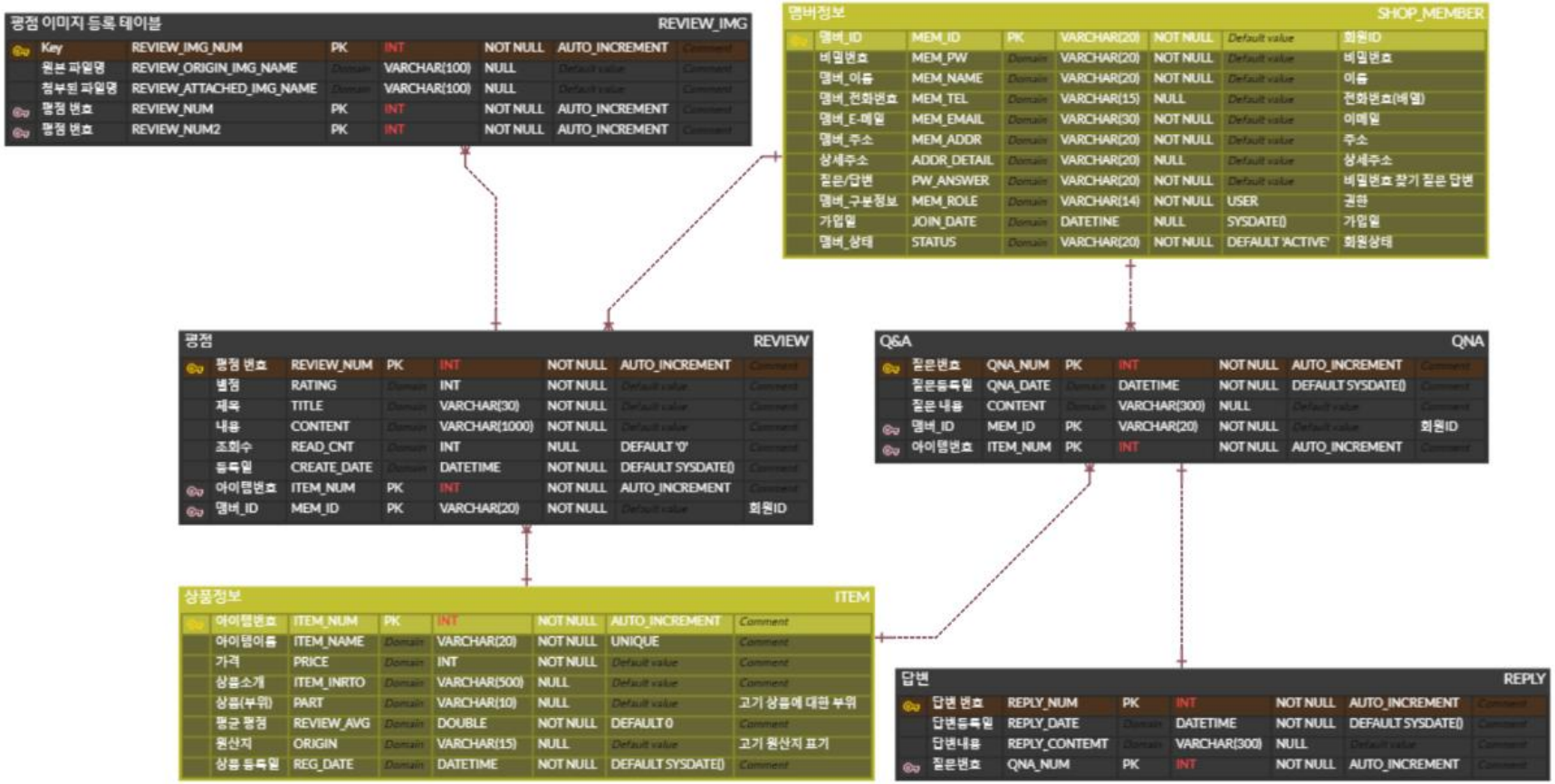
React-Native 기반 IoT 연동 Application 개발, 모바일 환경에서 실시간 센서 데이터 조회 기능 구현

프로젝트 구조 ERD – 상품 주문 모듈



프로젝트 구조 ERD – 회원 모듈 및 찜하기

프로젝트 구조 ERD – Q&A 및 평점 모듈



프로젝트 구조 ERD – IoT 기기 센서 측정 모듈

FAN 액추에이터						FAN_FUNCTION
Key	THV_ID	PK	INT	NOT NULL	AUTO_INCREMENT	*Comment*
온도	TEMPERATURE	*Domain*	FLOAT	NULL	*Default value*	온도 측정 값
습도	HUMIDITY	*Domain*	FLOAT	NULL	*Default value*	습도 측정 값
공기질	AIR_QUALITY	*Domain*	FLOAT	NULL	*Default value*	공기질 측정 값
측정 값 등록일	CREATE_DATE	*Domain*	DATETIME	NOT NULL	DEFAULT SYSDATE()	*Comment*

LED 액추에이터						ILLUMINANCE_FUNCTION
Key	Key	PK	INT	NOT NULL	AUTO_INCREMENT	*Comment*
조도	ILLUMINANCE	*Domain*	INT	NULL	*Default value*	조도측정 값
측정 값 등록일	CREATE_DATE	*Domain*	DATETIME	NOT NULL	DEFAULT SYSDATE()	*Comment*

주요 역할 및 기여

상품 목록 페이지

중복 코드 리팩토링	Home / 신상품 페이지 중복 로직을 컴포넌트화하여 32줄 코드 제거
Pagination 구현	대량 상품 목록의 가독성과 UI 개선

상품 상세 페이지

Outlet 데이터 전달	useOutletContext 적용하여 상세 페이지 데이터 안정적 전달

관리자 차트 페이지

데이터 차트 구성	기간 선택 기능(1~4주) 및 Chart.js 기반 최고/평균/최저 데이터 시각화

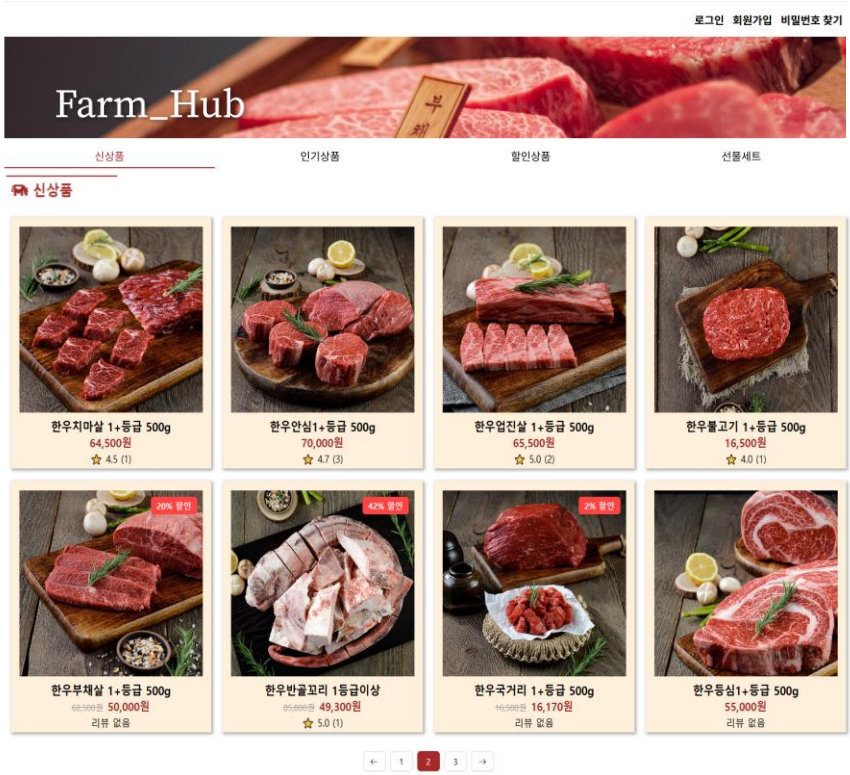
상품 목록 페이지

페이지 설명

- Home 화면에서 신상품, 인기상품, 할인상품이 조회됨
- 상위 탭을 누르거나, 아래 '더보기'버튼을 누르면 해당 상품 목록 페이지로 이동됨

링크

https://youtu.be/TCpzN9cTWgo



상품 목록 페이지

중복 코드 리팩토링

상품 목록 컴포넌트 분리로 유지보수성 향상

📌 개요

Home과 신상품 페이지에서 상품 목록 조회 로직이 중복되는 문제를 확인하여, 공통 컴포넌트를 분리함으로써 유지보수성을 개선함.

⚙️ 구현 내용

공통 로직을 별도 컴포넌트로 분리, props로 데이터 전달, 중복 코드 32줄 제거

💡 문제 해결

문제

Home은 신상품 8개, NewProductList는 전체 표시 필요

해결

useLocation Hook으로 현재 URL을 확인한 뒤, 삼항 연산자를 사용해 Home에서는 8건만 표시하고 목록 페이지에서는 전체 데이터를 표시하도록 분기함.

✨ 결과

코드 가독성 향상, 수정 시간 단축, 컴포넌트 재사용 이해도 향상

🗒 링크

https://sticky-chill-dd2.notion.site/11-279342983be3805dba73f80e8ba26a9a?source=copy_link

💻 코드

```python
import React from 'react'
import styles from './NewProductList.module.css'
import PageTitle from '../../../common/PageTitle'
import { useNavigate } from 'react-router'
import Menu from '../../../components/Menu'
import NewProducts from '../../../components/ProductList/NewProducts'

const NewProductList = () => {
  const nav = useNavigate();

  return (
    <div className={styles.container}>
      <Menu />
      <NewProducts />
    </div>
  )
}

export default NewProductList
```

📷 화면 구현

상품 목록 페이지

Pagination 구현

상품 목록 페이지 가독성 및 UI 개선

📌 개요

한 페이지에 50건 이상의 데이터가 조회되며 가독성이 떨어지는 문제를 확인하고, Pagination을 적용해 페이지당 8건만 표시하도록 개선하여 UI를 향상함.

⚙️ 구현 내용

react-js-pagination 라이브러리 활용, 페이지당 8개 항목, useState로 상태 관리

💡 문제 해결

문제

totalItems에 현재 페이지 데이터만 전달하여 페이지 수 계산 오류

해결

전체 배열 length를 totalItems로 전달

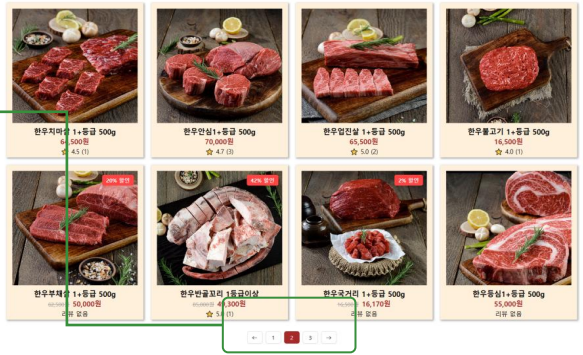
✨ 결과

가독성 향상, 페이지 이동 편의성 개선, 상태 관리 이해 증진

🔗 링크

https://sticky-chill-dd2.notion.site/16-280342983be381c1a6a6faefc20b0c3f?source=copy_link

📸 화면 구현



💻 코드

```python
urlInfo.pathname !== '/'
        &&
        <Pagination
          totalItems={newProducts.length}
          itemsPerPage={itemsPerPage}
          onPageChange={handlePageChange}
          currentPage={currentPage}
          nextLabel={<i class="bi bi-arrow-right"></i>}
          previousLabel={<i class="bi bi-arrow-left"></i>}
          color='brown'
```

```python
import React from 'react';
import ReactPaginate from 'react-paginate';
import styles from './Pagination.module.css';

const Pagination = ({
  totalItems,          // 전체 아이템 개수 (예: 100개)
  itemsPerPage = 10,   // 페이지당 보여줄 아이템 수 (기본값: 10개)
  onPageChange,        // 페이지 변경 시 호출될 함수 (부모 컴포넌트에서 전달)
  currentPage = 0,     // 현재 활성화된 페이지 (0부터 시작, 기본값: 0)
  previousLabel="이전", // "이전" 버튼에 표시될 텍스트 (기본값: "이전")
  nextLabel="다음",    // "다음" 버튼에 표시될 텍스트 (기본값: "다음")
  color = 'green',     // 페이지네이션 색상 테마 (기본값: 'green')
                       // 가능한 값: 'blue', 'green', 'red', 'brown', 'orange', 'gray'
  ...props             // 추가로 전달되는 모든 props (ReactPaginate 컴포넌트로 전달됨)
}) => {
  // 전체 아이템 수를 페이지당 아이템 수로 나누어 총 페이지 수 계산
  const pageCount = Math.ceil(totalItems / itemsPerPage);

  const handlePageChange = ({ selected }) => {
    onPageChange(selected); // 선택된 페이지 번호를 부모 컴포넌트로 전달
    // onPageChange함수는 구현할 페이지에 만들어놓은 페이지를 변경시켜줄 함수 자체
  };

  return (
    <ReactPaginate
      pageCount={pageCount}              // 총 페이지 수
      onPageChange={handlePageChange}    // 페이지 변경 핸들러 함수
      previousLabel={previousLabel}      // "이전" 버튼 라벨
      nextLabel={nextLabel}              // "다음" 버튼 라벨
      className={ `${styles.pagination} ${styles[color]}` }  // CSS 클래스 적용
      forcePage={currentPage}            // 현재 활성화할 페이지 강제 설정
      {...props}                         // 추가 props 전달
    />
  );
};

export default Pagination;
```

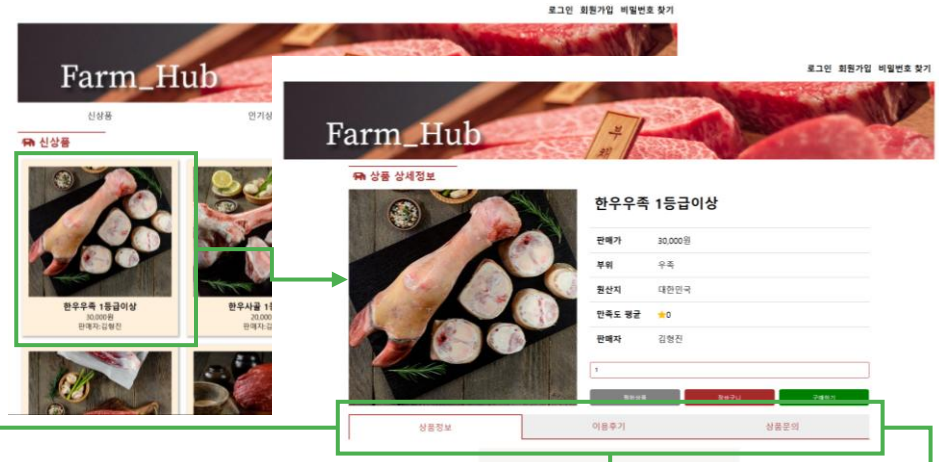
상품 상세 페이지

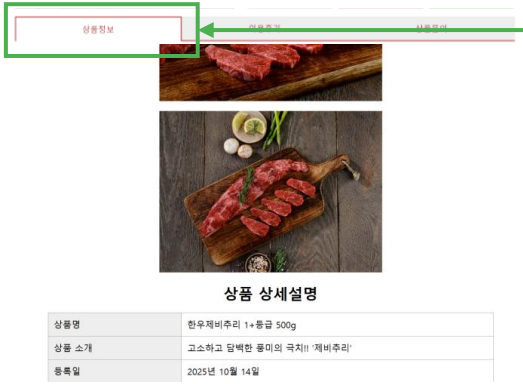
페이지 설명

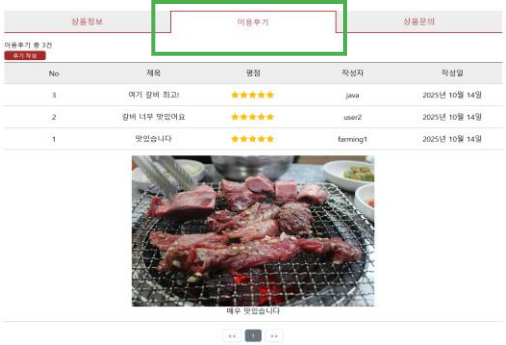
- 상품 목록 페이지에서 상품 영역을 클릭하면, 해당 상품 상세 페이지로 이동됨
- 상품 상세 페이지는 상단에, 상품 메인이미지와 설명이 적혀 있고, 하단에는 ‘상품정보’, ‘이용후기’, ‘상품문의’탭이 존재함

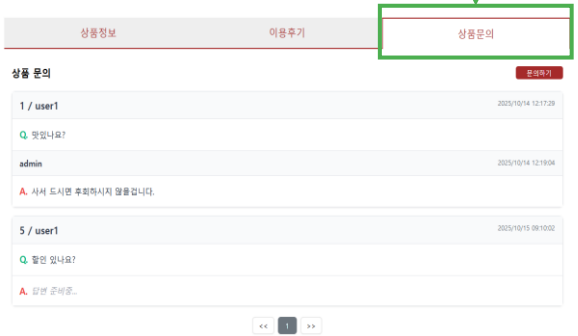
링크

https://youtu.be/kpr2b_3jrx8









상품 상세 페이지

Outlet 데이터 전달

useOutletContext를 활용한 상세 페이지 데이터 공유

📌 개요

상품 상세 페이지의 상단 정보와 하단 탭이 동일 데이터 필요. useOutletContext로 데이터 공유

⚙️ 구현 내용

React Router useOutletContext 활용, 부모에서 자식으로 데이터 전달

💡 문제 해결

문제

각 탭마다 API 재호출로 불필요한 네트워크 요청 발생

해결

useOutletContext로 데이터 한 번만 조회 후 공유

✨ 결과

API 호출 최소화, 페이지 전환 속도 개선, Context API 활용 경험

🔗 링크

https://sticky-chill-dd2.notion.site/9-277342983be380b69396dc2a30378c5b?source=copy_link

💻 코드

```react_router_outlet_context_usage.md
import React from 'react'
import styles from './ProductIntro.module.css'
import { useOutletContext } from 'react-router';
import dayjs from 'dayjs';

const ProductIntro = () => {
  const { itemDetail } = useOutletContext(); // 부모에서 보낸 context 받기

  return (
    <div className={styles.container}>
      <div className={styles.img_div}>
        {
          itemDetail.imgList &&
          itemDetail.imgList.map((img, i) => {
            if (img.isMain === 'N') {
              return (
                <img
                  src={`http://localhost:8080/upload/${img.attachedImgName}`}
                  className={styles.subImg}
                  key={i}
                />
              )
            }
          })
```

📸 화면 구현

상품 상세설명

상품명	한우제비추리 1+등급 500g
상품 소개	고소하고 담백한 풍미의 극치!! '제비추리'
등록일	2025년 10월 14일

관리자 차트 페이지

페이지 설명

- 축사데이터는 온도, 습도, 조도, 공기질 데이터가 출력됨
- 축사 데이터 화면에서는 1주전~4주전 데이터를 확인할 수 있음
- 데이터는 최고, 최저, 평균 데이터가 출력됨

링크

https://youtu.be/DG-MDDfSU40

관리자 페이지 사이드 메뉴

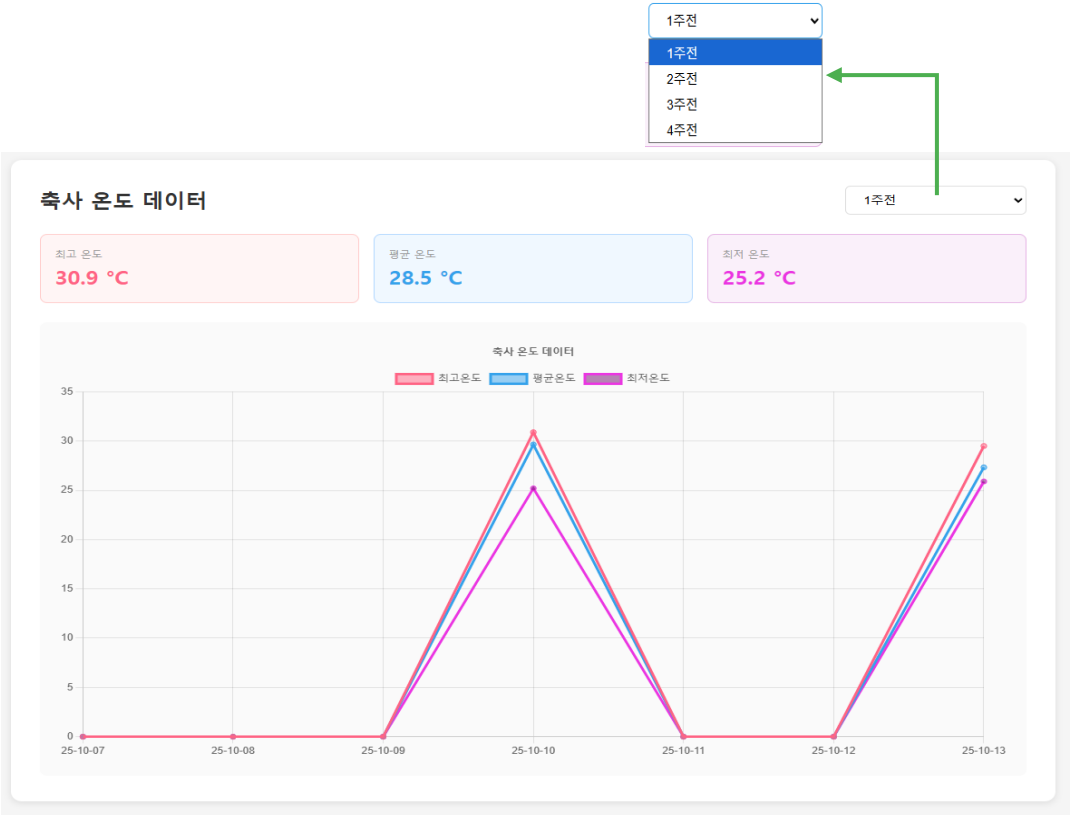
축사정보

🌡온도

💧습도

☁공기질

☀조도



관리자 차트 페이지

데이터 차트 구성

기간별 온도 데이터 시각화 및 통계 정보 표시

📌 개요

기간별 온도 변화를 Chart.js로 시각화, 최고/평균/최저 온도, 공기질 표시

⚙️ 구현 내용

Chart.js Line Chart 활용, 1-4주 기간 선택 기능, 통계 데이터 계산 및 표시

💡 문제 해결

문제

조도 데이터 조회 시 WHERE 절 누락으로 날짜 구분 없이 동일한 값만 반복 조회됨

해결

WHERE 절로 날짜별 필터링 추가, ORDER BY로 정렬하여 기간별 정확한 데이터 표시

✨ 결과

1~4주 데이터 비교와 통계 자동 계산을 통해 센서 상태를 빠르게 파악할 수 있도록 하고,
최고·평균·최저값을 시각적으로 구분해 확인 속도를 더욱 높임.

🗒 링크

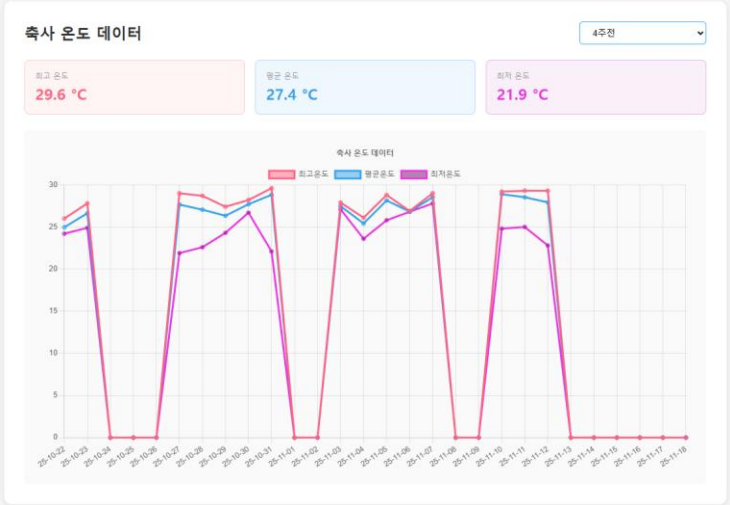
https://sticky-chill-dd2.notion.site/17-288342983be381bfaef5fc099e5c6197?source=copy_link
https://sticky-chill-dd2.notion.site/18-28d342983be380119aeede978dc4e642?source=copy_link

💻 코드

farm-mapper.xml

```xml
<!--온도 데이터 조회-->
<select id="getTemperatureData" resultMap="farm">
    <foreach collection="dateRange" item="each" separator="UNION ALL">
        SELECT
        IFNULL(CREATE_DATE, ADDDATE(CURRENT_DATE(), -#{each})) CREATE_DATE
        , IFNULL(AVG(TEMPERATURE), 0) AS AVG_TEMP
        , IFNULL(MAX(TEMPERATURE), 0) AS MAX_TEMP
        , IFNULL(MIN(TEMPERATURE), 0) AS MIN_TEMP
        FROM fan_function
        WHERE TO_CHAR(CREATE_DATE, 'YYYY-MM-DD') = ADDDATE(CURRENT_DATE(), -#{each})
    </foreach>
    ORDER BY CREATE_DATE ASC
</select>
```

📸 화면 구현



앞으로의 목표

- 복습 중심의 학습 습관을 유지하겠습니다
- 이해가 어려운 부분은 Notion에 정리하고, 스스로 해결하며 익히겠습니다
- 온라인 자료를 통해 부족한 부분을 보완하겠습니다
- 단순히 기능 구현을 넘어, 문제의 원인을 분석하고 개선하는 개발자가 되겠습니다
- 꾸준함과 기록을 바탕으로, 팀과 함께 성장하는 개발자로 발전하겠습니다

감사합니다